National University of Computer and Emerging Sciences

AL-2002: Artificial Intelligence Lab
Lab Manual 09

Department of Computer Science
FAST-NU, Lahore, Pakistan

Table of Contents

Objectives 3
1 What is Machine learning? 3
1.1 Types of Machine Learning 3
1.2 Supervised Learning 4
2 What is Scikit-learn? 5
2.1 Basic Workflow in Scikit-learn 7
3 Classification 9
3.1 Import Libraries 10
3.2 Load the Dataset 10
3.3 Split Data (Train/Test) 10
3.4 Feature Scaling 10
3.5 Train the Classifier 11
3.6 Make Predictions 11
3.7 Evaluate the Model 11
3.8 Trying Another Classifier 12
4 Regression 12
4.1 Import Libraries 12
4.2 Load the Dataset 13
4.3 Split Data (Train/Test) 13
4.4 Feature Scaling 13
4.5 Train the Regression Model 13
4.6 Make Predictions 14
4.7 Evaluate the Model 14
4.8 Trying Another Regression Model 14

Objectives

After performing this lab, students shall be able to understand the following concepts and applications:

- ✔ Use of scikitt-learn for classification and regression
- ✔ Basic model evaluation

1 What is Machine learning?

Machine Learning (ML) is a field of computer science that enables computers to learn patterns from data and make predictions or decisions without being explicitly programmed for each task. Instead of manually writing rules, we allow the system to learn those rules automatically from examples.

Traditional Programming

- Input data + Fixed rules = Output
- Rules are written by humans

Machine Learning

- Input data + Correct outputs = Model learns rules
- Rules are learned automatically from data

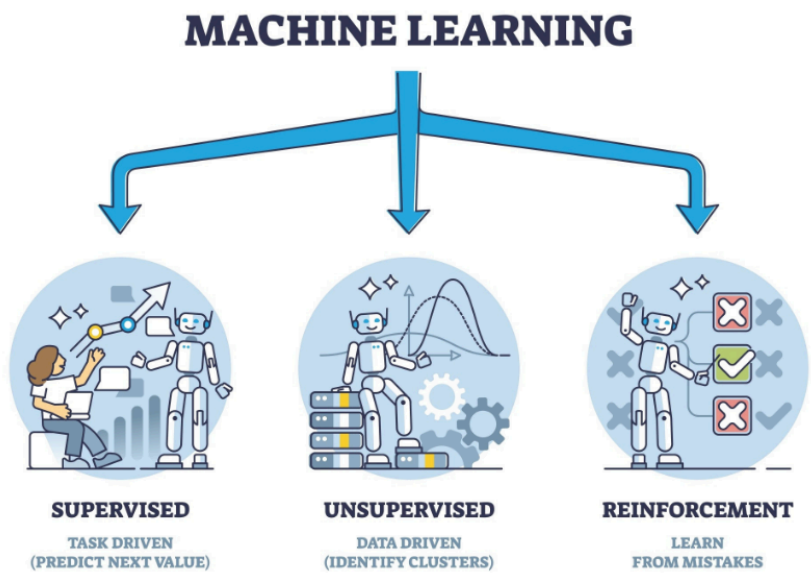
Some real world examples of machine learning include:

- Spam email detection
- Medical diagnosis
- House price prediction
- Credit card fraud detection
- Recommendation systems

1.1 Types of Machine Learning

Machine learning is generally divided into three categories:

Type	Description	Example
Supervised Learning	Data includes correct answers	Predict exam score
Unsupervised Learning	No labeled output	Customer grouping
Reinforcement Learning	Learning via rewards	Game playing AI



1.2 Supervised Learning

Supervised learning means that we have input variables (features): X and an output variable (target/label): y. The model learns a function:

$$f(X) = y$$

Key Terms

- Feature: Input variable (e.g., house size)
- Target: Output variable (e.g., house price)
- Dataset: Collection of samples
- Model: Algorithm that learns patterns
- Training: Learning from known data
- Testing: Evaluating on unseen data

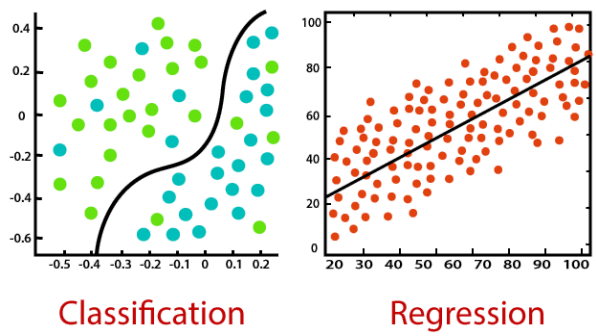
Supervised learning has two main categories. Each of them is explained below.

1.2.1 Regression

Regression is a type of supervised learning used when the target variable is continuous. A continuous variable can take any numerical value within a range. For example, predicting house prices, estimating temperatures, forecasting sales revenue, or estimating a student's final exam score are all regression problems.

1.2.2 Classification

Classification is a type of supervised learning used when the target variable is categorical. In classification problems, the goal is to assign an input example to one of several predefined classes. For example, determining whether an email is spam or not spam, predicting whether a patient has a disease or not, or classifying images of animals into categories are all classification tasks.



2 What is Scikit-learn?

Scikit-learn (also written as sklearn) is an open-source machine learning library for Python. It provides simple and efficient tools for building, training, and evaluating machine learning models.

It is built on top of:

- NumPy (numerical computation)
- Pandas (data handling)
- Matplotlib (visualization)

Before libraries like Scikit-learn, implementing machine learning algorithms required writing complex mathematical code from scratch.

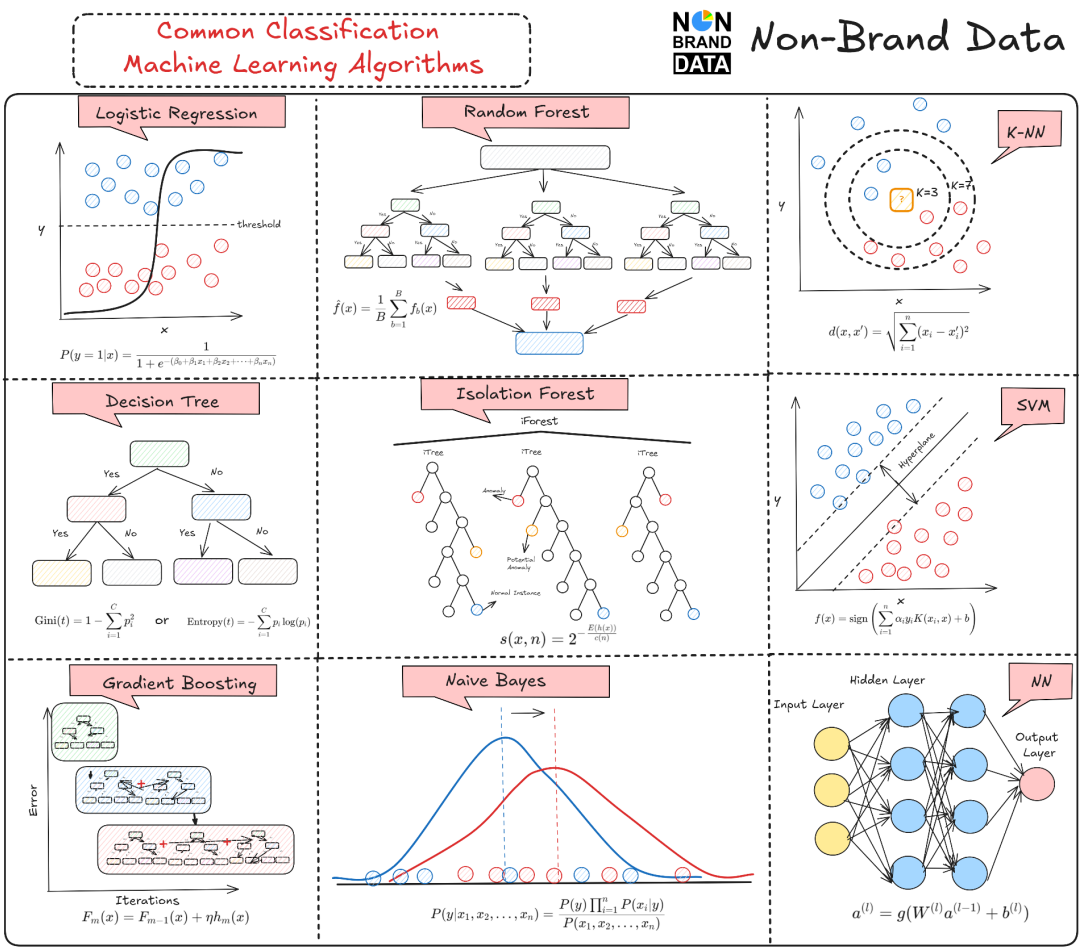
With Scikit-learn, we can:

- Load datasets easily
- Preprocess data
- Train models in a few lines of code
- Evaluate performance
- Compare different algorithms

Scikit-learn supports many machine learning tasks such as:

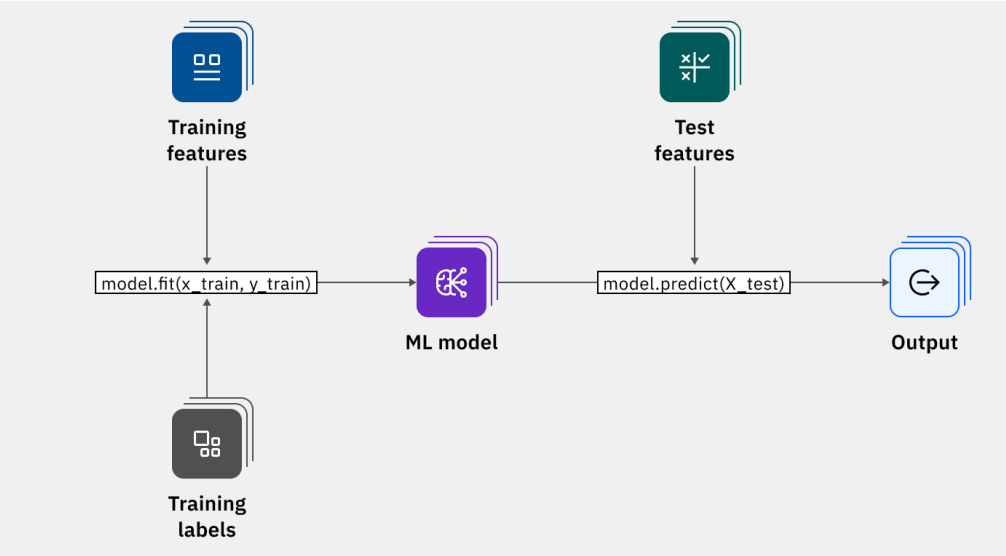
- Supervised Learning
    - Linear Regression

    - Logistic Regression
    - K-Nearest Neighbors
    - Decision Trees
    - Random Forest
    - Support Vector Machines
- Unsupervised Learning
    - K-Means Clustering
    - Hierarchical Clustering
    - PCA (Dimensionality Reduction)
- Preprocessing Tools
    - Scaling features
    - Encoding categorical variables
    - Handling missing values
- Model Evaluation
    - Accuracy
    - Precision, Recall
    - Confusion Matrix
    - Cross-validation

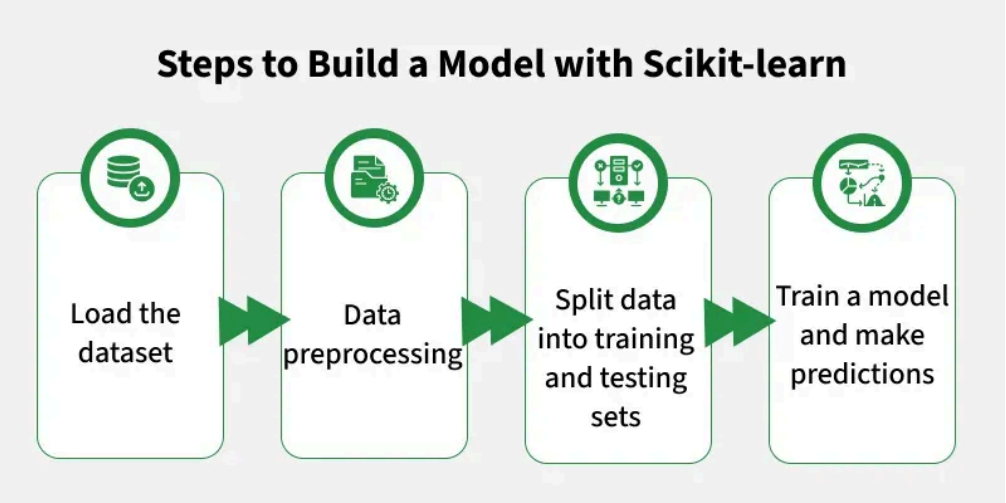


All models in Scikit-learn follow the same structure

Method	Purpose
fit()	Train the model
predict()	Make predictions
score()	Evaluate performance



2.1 Basic Workflow in Scikit-learn



We have already covered the first 2 steps in previous labs.

2.1.1 Split Data (Train/Test)

In machine learning, our goal is not just to build a model that performs well on known data, but one that performs well on new, unseen data.

If we train and test the model using the same dataset, the model may memorize the data and the performance may look very high but it may fail in real-world scenarios.

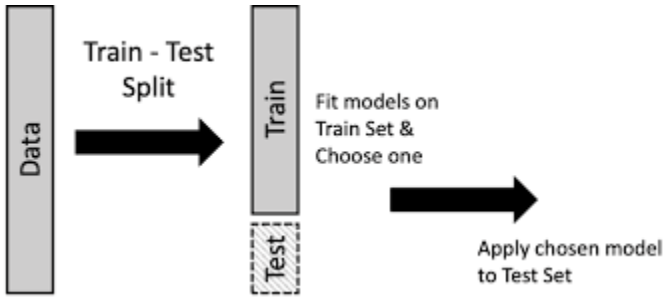
To avoid this, we divide the dataset into two parts:

- **Training Set:** The training set is used to:
    - Teach the model
    - Learn patterns
    - Estimate parameters (weights)

  During training, the model:

    - Looks at input features (X)
    - Compares predictions with actual outputs (y)
    - Adjusts its parameters to reduce error
- **Testing Set:** The testing set is used to:
    - Evaluate model performance
    - Measure generalization ability
    - Simulate real-world prediction

The model must never learn from the test data. The test data must remain unseen during training.



Suppose we have 100 student records. If we split 80–20:

- 80 records: Training set
- 20 records: Testing set

The model learns from the 80 students and is evaluated on the 20 new students.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,test_size=0.2,
random_state=42)
```

Parameter	Meaning
X	Features
y	Target
test_size=0.2	20% data used for testing
random_state=42	Ensures reproducibility

When splitting data:

- The split is random.
- Different runs may give different results.

By setting random_state=42, we ensure:

- Same split every time
- Reproducible results
- Fair comparison between models

2.1.2 Model Training

We will be covering 2 types of models: classification and regression.

3 Classification

In this section, we will go through an example of how we can train and test a classification model step by step.

3.1 Import Libraries

```
import numpy as np
import pandas as pd

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
```

3.2 Load the Dataset

```
iris = load_iris()

X = iris.data       # Features
y = iris.target     # Labels

print(X.shape)  # (150 samples, 4 features)
print(y.shape)  # (150,)
```

3.3 Split Data (Train/Test)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

3.4 Feature Scaling

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

3.5 Train the Classifier

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

```
LogisticRegression
LogisticRegression()
```

We are using logistic regression as a classifier here. You can explore other classifiers here: https://scikit-learn.org/stable/supervised_learning.html

What happens internally:

- The model learns decision boundaries
- It finds weights that best separate classes

3.6 Make Predictions

```
y_pred = model.predict(X_test)
```

3.7 Evaluate the Model

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 1.0
```

```
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

3.8 Trying Another Classifier

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)

print("KNN Accuracy:", accuracy_score(y_test, y_pred_knn)))
```

```
KNN Accuracy: 1.0
```

4 Regression

In this section, we will go through an example of how we can train and test a regression model step by step.

4.1 Import Libraries

```
import numpy as np
import pandas as pd

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

4.2 Load the Dataset

```
data = fetch_california_housing()

X = data.data     # Features
y = data.target   # Target

print(X.shape)  # (20640 samples, 8 features)
print(y.shape)  # (20640,)
```

4.3 Split Data (Train/Test)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

4.4 Feature Scaling

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

4.5 Train the Regression Model

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
LinearRegression
LinearRegression()
```

We are using linear regression as a regressor here. You can explore other regressors here:

https://scikit-learn.org/stable/supervised_learning.html

What happens internally:

- Model finds weights for each feature
- Minimizes the difference between predicted and actual values

4.6 Make Predictions

```
y_pred = model.predict(X_test)
```

4.7 Evaluate the Model

```
mse = mean_squared_error(y_test, y_pred)
print("MSE:", mse)
```

```
MSE: 0.5558915986952442
```

```
rmse = np.sqrt(mse)
print("RMSE:", rmse)
```

```
RMSE: 0.7455813830127763
```

```
r2 = r2_score(y_test, y_pred)
print("R² Score:", r2)
```

```
R² Score: 0.575787706032451
```

4.8 Trying Another Regression Model

```
from sklearn.tree import DecisionTreeRegressor

tree_model = DecisionTreeRegressor(random_state=42)
tree_model.fit(X_train, y_train)
```

```
y_pred_tree = tree_model.predict(X_test)

print("Decision Tree R² Score:", r2_score(y_test, y_pred_tree))
```

```
Decision Tree R² Score: 0.6230424613065773
```